

Computer Vision with OpenCV – Lab

Experiment Codes

1(a). Convert Image to Grayscale

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap="gray")
plt.axis("off")
plt.show()
```

1(b). Gaussian Blur

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
blur = cv2.GaussianBlur(img_rgb, (15, 15), 0)

plt.imshow(blur)
plt.axis("off")
plt.show()
```

1(c). Canny Outline

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)

plt.imshow(edges, cmap="gray")
plt.axis("off")
plt.show()
```

1(d). Dilation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
```

```
kernel = np.ones((5, 5), np.uint8)
dilated = cv2.dilate(img, kernel, iterations=1)
```

```
plt.imshow(dilated, cmap="gray")
plt.axis("off")
plt.show()
```

1(e). Erosion

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
img = cv2.imread("/content/image.jpg", 0)
kernel = np.ones((5, 5), np.uint8)
eroded = cv2.erode(img, kernel, iterations=1)
```

```
plt.imshow(eroded, cmap="gray")
plt.axis("off")
plt.show()
```

2. Video in Slow and Fast Motion

```
import cv2
```

```
cap = cv2.VideoCapture("/content/video.mp4")
delay = 30
```

```
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
```

```
    cv2.imshow("Video", frame)
    key = cv2.waitKey(delay) & 0xFF
```

```
    if key == ord("s"):
        delay = 100
    elif key == ord("n"):
        delay = 30
    elif key == ord("f"):
        delay = 1
    elif key == ord("q"):
        break
```

```
cap.release()
cv2.destroyAllWindows()
```

3. Webcam Video Capture

```
import cv2
```

```

cap = cv2.VideoCapture(0)
delay = 30

while True:
    ret, frame = cap.read()
    if not ret:
        break

    cv2.imshow("Webcam", frame)
    key = cv2.waitKey(delay) & 0xFF

    if key == ord("s"):
        delay = 100
    elif key == ord("n"):
        delay = 30
    elif key == ord("f"):
        delay = 1
    elif key == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()

```

4. Image Scaling

```

import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

smaller = cv2.resize(img, None, fx=0.5, fy=0.5)
bigger = cv2.resize(img, None, fx=2, fy=2)

plt.figure(figsize=(15, 5))
for i, (title, im) in enumerate([("Original", img), ("Smaller", smaller), ("Bigger", bigger)], 1):
    plt.subplot(1, 3, i)
    plt.imshow(im)
    plt.title(title)
    plt.axis("off")
plt.show()

```

5. Image Rotation

```

import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

clockwise = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
counter = cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)

```

```
plt.figure(figsize=(15, 5))
for i, (title, im) in enumerate([("Original", img), ("Clockwise", clockwise), ("Counter Clockwise", counter)], 1):
    plt.subplot(1, 3, i)
    plt.imshow(im)
    plt.title(title)
    plt.axis("off")
plt.show()
```

6. Image Translation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
rows, cols = img.shape[:2]

M = np.float32([[1, 0, 100], [0, 1, 50]])
translated = cv2.warpAffine(img, M, (cols, rows))

plt.imshow(cv2.cvtColor(translated, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

7. Affine Transformation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
rows, cols = img.shape[:2]

pts1 = np.float32([[50, 50], [200, 50], [50, 200]])
pts2 = np.float32([[10, 100], [200, 50], [100, 250]])

M = cv2.getAffineTransform(pts1, pts2)
result = cv2.warpAffine(img, M, (cols, rows))

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

8. Perspective Transformation on Image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
rows, cols = img.shape[:2]
```

```

pts1 = np.float32([[0, 0], [cols-1, 0], [0, rows-1], [cols-1, rows-1]])
pts2 = np.float32([[100, 50], [cols-100, 50], [50, rows-50], [cols-50, rows-50]])

M = cv2.getPerspectiveTransform(pts1, pts2)
result = cv2.warpPerspective(img, M, (cols, rows))

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()

```

9. Perspective Transformation on Video

```

import cv2
import numpy as np

cap = cv2.VideoCapture("/content/video.mp4")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    rows, cols = frame.shape[:2]
    pts1 = np.float32([[0, 0], [cols-1, 0], [0, rows-1], [cols-1, rows-1]])
    pts2 = np.float32([[100, 50], [cols-100, 50], [50, rows-50], [cols-50, rows-50]])

    M = cv2.getPerspectiveTransform(pts1, pts2)
    result = cv2.warpPerspective(frame, M, (cols, rows))

    cv2.imshow("Perspective Video", result)
    if cv2.waitKey(25) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()

```

10. Homography Matrix Transformation

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
rows, cols = img.shape[:2]

src = np.float32([[0, 0], [cols-1, 0], [cols-1, rows-1], [0, rows-1]])
dst = np.float32([[100, 50], [cols-100, 30], [cols-50, rows-50], [50, rows-30]])

H, status = cv2.findHomography(src, dst)
result = cv2.warpPerspective(img, H, (cols, rows))

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))

```

```
plt.axis("off")
plt.show()
```

11. Direct Linear Transformation (DLT)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
rows, cols = img.shape[:2]

src = np.array([[0, 0], [cols-1, 0], [cols-1, rows-1], [0, rows-1]], dtype=np.float32)
dst = np.array([[50, 50], [cols-50, 30], [cols-30, rows-50], [30, rows-30]], dtype=np.float32)

A = []
for i in range(4):
    x, y = src[i]
    u, v = dst[i]
    A.append([-x, -y, -1, 0, 0, 0, x*u, y*u, u])
    A.append([0, 0, 0, -x, -y, -1, x*v, y*v, v])

A = np.array(A)
_, _, Vt = np.linalg.svd(A)
H = Vt[-1].reshape(3, 3)
H = H / H[2, 2]

result = cv2.warpPerspective(img, H, (cols, rows))

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

12. Canny Edge Detection

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
edges = cv2.Canny(img, 100, 200)

plt.imshow(edges, cmap="gray")
plt.axis("off")
plt.show()
```

13. Sobel Edge Detection – X Axis

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
```

```
plt.imshow(sobel_x, cmap="gray")
plt.axis("off")
plt.show()
```

14. Sobel Edge Detection – Y Axis

```
import cv2
import matplotlib.pyplot as plt
```

```
img = cv2.imread("/content/image.jpg", 0)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
```

```
plt.imshow(sobel_y, cmap="gray")
plt.axis("off")
plt.show()
```

15. Sobel Edge Detection – XY Axis

```
import cv2
import matplotlib.pyplot as plt
```

```
img = cv2.imread("/content/image.jpg", 0)
```

```
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1)
```

```
sobel_xy = cv2.addWeighted(
    cv2.convertScaleAbs(sobel_x), 0.5,
    cv2.convertScaleAbs(sobel_y), 0.5, 0
)
```

```
plt.imshow(sobel_xy, cmap="gray")
plt.axis("off")
plt.show()
```

16. Sharpening – Laplacian Negative Center

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
img = cv2.imread("/content/image.jpg")
```

```
kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])
result = cv2.filter2D(img, -1, kernel)
```

```
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

17. Sharpening – Laplacian with Diagonal Neighbors

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")

kernel = np.array([[ -1, -1, -1], [-1, 9, -1], [-1, -1, -1]])
result = cv2.filter2D(img, -1, kernel)

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

18. Sharpening – Positive Center Coefficient

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg").astype(np.float32)

laplacian = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]])
filtered = cv2.filter2D(img, cv2.CV_32F, laplacian)
sharp = np.clip(img - filtered, 0, 255).astype(np.uint8)

plt.imshow(cv2.cvtColor(sharp, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

19. Unsharp Masking

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
blur = cv2.GaussianBlur(img, (9, 9), 10)
sharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)

plt.imshow(cv2.cvtColor(sharp, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

20. High-Boost Filtering

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg").astype(np.float32)
blur = cv2.GaussianBlur(img, (9, 9), 10)
mask = img - blur
```



```
k = 2
high_boost = np.clip(img + k * mask, 0, 255).astype(np.uint8)

plt.imshow(cv2.cvtColor(high_boost, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

21. Gradient Masking

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)

sobel_x = cv2.Sobel(img, cv2.CV_32F, 1, 0)
sobel_y = cv2.Sobel(img, cv2.CV_32F, 0, 1)
gradient = cv2.magnitude(sobel_x, sobel_y)
gradient = cv2.convertScaleAbs(gradient)

plt.imshow(gradient, cmap="gray")
plt.axis("off")
plt.show()
```

22. Insert Watermark into Image

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
watermarked = img.copy()

cv2.putText(
    watermarked, "VISHNU", (50, 100),
    cv2.FONT_HERSHEY_SIMPLEX, 2,
    (0, 0, 255), 3
)

plt.imshow(cv2.cvtColor(watermarked, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

23. Cropping, Copying and Pasting

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")
crop = img[50:250, 50:250]

result = img.copy()
result[300:500, 300:500] = crop
```

```
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

24. Boundary Detection Using Convolution Kernel

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)

kernel = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
])

boundary = cv2.filter2D(img, -1, kernel)

plt.imshow(boundary, cmap="gray")
plt.axis("off")
plt.show()
```

25. Morphological Erosion

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
kernel = np.ones((5, 5), np.uint8)
result = cv2.erode(img, kernel, iterations=1)

plt.imshow(result, cmap="gray")
plt.axis("off")
plt.show()
```

26. Morphological Dilation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
kernel = np.ones((5, 5), np.uint8)
result = cv2.dilate(img, kernel, iterations=1)

plt.imshow(result, cmap="gray")
plt.axis("off")
plt.show()
```

27. Morphological Opening

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)

plt.imshow(result, cmap="gray")
plt.axis("off")
plt.show()
```

28. Morphological Closing

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)

plt.imshow(result, cmap="gray")
plt.axis("off")
plt.show()
```

29. Morphological Gradient

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)

plt.imshow(result, cmap="gray")
plt.axis("off")
plt.show()
```

30. Top Hat Transformation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)

plt.imshow(result, cmap="gray")
```

```
plt.axis("off")
plt.show()
```

31. Black Hat Transformation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg", 0)
kernel = np.ones((5, 5), np.uint8)
result = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)

plt.imshow(result, cmap="gray")
plt.axis("off")
plt.show()
```

32. Watch Recognition Using Object Detection

```
import cv2

img = cv2.imread("/content/watch.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 50, 150)

contours, _ = cv2.findContours(
    edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

for contour in contours:
    if cv2.contourArea(contour) > 1000:
        x, y, w, h = cv2.boundingRect(contour)
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 3)
        cv2.putText(img, "Watch", (x, y-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8,
                    (0, 255, 0), 2)

cv2.imshow("Watch Recognition", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

33. Play Video in Reverse Mode

```
import cv2

cap = cv2.VideoCapture("/content/video.mp4")
frames = []

while True:
    ret, frame = cap.read()
    if not ret:
```

```

        break
    frames.append(frame)

cap.release()

for frame in reversed(frames):
    cv2.imshow("Reverse Video", frame)
    if cv2.waitKey(30) & 0xFF == ord("q"):
        break

cv2.destroyAllWindows()

```

34. Face Detection Using OpenCV

```

import cv2

img = cv2.imread("/content/image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)

faces = face_cascade.detectMultiScale(
    gray, scaleFactor=1.1, minNeighbors=5
)

for x, y, w, h in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 3)

cv2.imshow("Face Detection", img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

35. Vehicle Detection in a Video

```

import cv2

car_cascade = cv2.CascadeClassifier("cars.xml")
cap = cv2.VideoCapture("/content/vehicles.mp4")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    cars = car_cascade.detectMultiScale(gray, 1.1, 3)

    for x, y, w, h in cars:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(frame, "Vehicle", (x, y-10),

```

```

        cv2.FONT_HERSHEY_SIMPLEX, 0.7,
        (0, 255, 0), 2)

cv2.imshow("Vehicle Detection", frame)

if cv2.waitKey(30) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()

```

36. Draw Rectangle and Extract Object

```

import cv2
import matplotlib.pyplot as plt

img = cv2.imread("/content/image.jpg")

x, y, w, h = 100, 100, 200, 200

cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 3)
object_image = img[y:y+h, x:x+w]

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Rectangle Object")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(object_image, cv2.COLOR_BGR2RGB))
plt.title("Extracted Object")
plt.axis("off")

plt.show()

```

Important Note

Change `"/content/image.jpg"`, `"/content/video.mp4"`, and other file paths to match your actual files.

For Experiment 35, the required vehicle cascade XML file (cars.xml) must be available in the working directory.